Compresión con Árboles de Huffman

José Rodolfo Godínez
José Alexander Brenes

26 de mayo de 2024

Índice

1. Objetivo **2**

2. Antecedentes **2**

3. Especificación del problema **2**
3.1. Descripción del algoritmo 3
3.2. Entrada 3
3.2.1. Entrada del compresor 3
3.2.2. Entrada del descompresor 4
3.3. Salida 4
3.3.1. Compresor 4
3.3.2. Descompresor 4
3.4. Documentación 4

4. Sugerencias **5**
4.1. Uso de argumentos en Python 5
4.2. Arreglos de bits en Python 6

5. Aspectos Generales **6**

1. Objetivo

Esta tarea tiene como objetivo que el estudiante demuestre capacidad para la utilización de árboles, listas y recursión en particular programadas utilizando Python3 en el manejo y administración de archivos.

2. Antecedentes

El código de Huffman es un tipo de código de prefijos que se utiliza para comprimir datos sin pérdida (es decir, que se puede reconstruir el archivo original tal cual fue comprimido). El algoritmo fue desarrollado por David A. Huffman y fue publicado en 1952 (Wikipedia, s.f.).

3. Especificación del problema

Usted debe escribir en Python dos programas, compresor y descompresor. El compresor debe recibir como argumento el nombre de un archivo de texto plano que se desea comprimir. El programa deberá generar tres archivos a partir de este: un archivo comprimido con el mismo nombre pero la extensión .huff, un archivo de texto plano con la tabla de códigos que debe tener el mismo nombre con la extensión .table, y el tercer archivo, con la extensión .stats, debe tener algunas estadísticas de la compresión:

1. Altura del árbol
2. Anchura del árbol
3. Cantidad de nodos por nivel
4. Tabla de frecuencias original (sólo aquellas mayores a cero)

Por otro lado, el programa descompresor debe recibir como argumento el nombre del archivo comprimido, el nombre del archivo con la tabla de códigos y el nombre del archivo destino en el que se desea guardar el resultado. El programa deberá cargar el archivo comprimido y revertir el proceso de conversión exportando el archivo original.

Importante: El archivo comprimido debe ser de menor o igual tamaño que el archivo original.

Ejemplo:

```
$ python3 compresor.py file.txt
file.txt.huff file.txt.table file.txt.stats

$ python3 descompresor.py file.txt.huff file.txt.table archivo.txt
archivo.txt descomprimido con éxito
```

3.1. Descripción del algoritmo

El algoritmo primero calcula la frecuencia de cada byte (caracteres en el caso de un archivo de texto plano). Cada carácter lo representa como un nodo hoja de un árbol y crea una lista ordenada de nodos, de menor a mayor. Posteriormente extrae los dos nodos con frecuencia más baja y les asigna de padre un nodo nuevo sin símbolo que tiene como frecuencia la suma de sus hijos. Este nodo se agrega a la lista, respetando el orden por frecuencias. Este proceso se repite hasta que la lista tenga un único nodo, que será la raíz de un árbol binario, el árbol de Huffman.

Una vez obtenido el árbol, cada hoja de éste será conformada por uno de los símbolos y su respectiva frecuencia, y los nodos interiores serán padres con la suma de las frecuencias de sus hijos inmediatos.

La característica de los árboles es que a partir de su nodo fuente (la raíz), se cuenta con una ruta única para cada uno de los nodos, lo que también ocurre con las hojas (bytes del archivo original) y esta ruta será más corta cuanto más cerca de la raíz se encuentra, es decir, será más corta cuanto más frecuente sea.

Una vez halladas las rutas para cada hoja, se procede a asignarles un 0 para izquierda y un 1 para derecha (puede ser al revés) y se genera una tabla de códigos binarios únicos para cada símbolo del archivo. Finalmente se debe de reescribir el archivo, sólo que reemplazando la aparición de cada símbolo por su código (probablemente de menor longitud de un byte, pero si fuera mayor, habría otros más frecuentes, más cortos), lo que hace el archivo comprimido más pequeño. Adicionalmente se debe guardar la tabla de símbolos, con sus códigos respectivos, de forma que el árbol se pueda reconstruir y por tanto el archivo se pueda descomprimir.

En el enlace `https://youtu.be/apcCVfXfcqE` puede apreciarse el algoritmo con un ejemplo práctico.

3.2. Entrada

3.2.1. Entrada del compresor

Un archivo cualquiera.

3.2.2. Entrada del descompresor

Dos archivos generados por el compresor.

3.3. Salida

3.3.1. Compresor

Los tres archivos definidos anteriormente.

3.3.2. Descompresor

El archivo original con el nombre que tiene como tercer parámetro (puede que el archivo original haya sido borrado de la computadora).

3.4. Documentación

Deberá aportar un documento PDF en conjunto con su solución. Debe generar el documento usando LaTeX y deberá tener la siguiente estructura:

1. **Portada (5pts)**: Debe incluir el nombre de la institución, el nombre del departamento, el nombre de la carrera, el curso, número del grupo (grupo 21), el título del documento, los autores, la fecha y el lugar.
2. **Introducción (5pts)**: Resuma brevemente el documento, máximo una página, no incluya conclusiones en la introducción.
3. **Desarrollo**
   - *a*) Descripción del problema (5pts)
   - *b*) Investigación corta (30pts)
     1) Biografía de David A. Huffman
     2) Compresión de archivos con pérdida y sin pérdida
     3) Descripción de la estructura abstracta Trie
     4) Descripción de la biblioteca bitarray
     5) Descripción de las funciones `bitarray`, `tofile` y `fromfile` de bitarray.
   - *c*) Descripción de la solución (40pts): Descripción de cada función implementada donde se incluyan los siguientes datos:
     1) Nombre: Nombre de la función
     2) Módulo: Archivo en que se encuentra

3) Parámetros: Para cada parámetro incluir el nombre, tipo y propósito
4) Retorno: Indicar el tipo y descripción breve
5) Resumen: Descripción breve de la función

4. **Problemáticas y limitaciones (5pts)**: Indique qué aspectos del proyecto no pudo resolver. No documente aquellos aspectos en los cuales tuvo dificultades, pero pudo resolver.

5. **Conclusiones (10pts)**: Sintetice en dos o tres párrafos los aprendizajes obtenidos del proyecto. La redacción de las conclusiones debe ser impersonal y comprender contenido relacionado directamente con el proyecto. Por ejemplo, una conclusión de la redacción de este enunciado puede ser: "Sharelatex es una herramienta simple de usar que permite al usuario editar documentos de LaTeX en línea sin necesidad de instalar aplicaciones en la computadora personal."

6. **Referencias bibliográficas**: Debe presentar las referencias usadas en formato APA v.6. Recuerde que un error en el uso de citas o referencias bibliográficas puede representar la anulación de la entrega.

4. Sugerencias

4.1. Uso de argumentos en Python

El uso de argumentos de programa es una práctica muy común que permite encadenar los resultados de diversos programas en secuencia. Por ejemplo:

```
# Así se vería en terminal un programa que requiere la entrada
python3 compresor.py
archivo.txt

# Por otro lado, así se vería un programa que usa argumentos
python3 compresor.py archivo.txt
```

Nótese como en la segunda versión el nombre del archivo se incluye en la misma línea de comando en la cual se hace el llamado al programa. Esto permite encadenar diversos programas, usar los resultados de uno como insumos del siguiente.

En Python se utiliza la biblioteca nativa `sys` para leer los argumentos del programa. Los argumentos se almacenan en la variable `argv` de `sys`, la

cual es una lista. Esta biblioteca no requiere instalarse ya que es parte de Python.

```
import sys

if len(sys.argv) == 0 :
    print("El programa no tiene argumentos")
else :
    print("Primer Argumento:", sys.argv[0])
```

4.2. Arreglos de bits en Python

La biblioteca bitarray permite crear arreglos de bits en python. A continuación se presenta un ejemplo de cómo se crea un archivo binario a partir de una secuencia de bits:

```
import bitarray as bit
# Crea una secuencia de bits
bits = bit.bitarray('00001111')

# Almacena la secuencia en un archivo binario
with open('binary.bin', wb) as bf:
    bits.tofile(bf)
```

Además es posible leer un archivo binario y convertirlo a una secuencia de bits:

```
import bitarray as bit

# Crea una secuencia de bits vacía
bits = bit.bitarray()

# Carga la secuencia del archivo binario a la variable bits
with open('binary.bin', rb) as bf:
    bits.fromfile(bf)
```

5. Aspectos Generales

- Toda la programación debe realizarse en Python3 y será revisado sobre Linux.

- La fecha de entrega de este proyecto es el 17 de junio de 2024 con citas de revisión.
- El trabajo puede realizarse en parejas o individualmente.